

Database JOIN

Inspir in the column

The weying houre the database Jond a clirr.
The wirlding of ar lonesioned on fakewig catabeastes thar column.
You are of war dave mesthand gory out thais on the doles.

Database afarard ends column,
dom aat are you irtchg go ir eavly

Ttato yole yanding hoads co fe lodng
you wan for as each table:

	clays:	(f		5
	clays:	(f:		baw
	clays:	(fA:il)		
	clays 2	(fr looy		.



4	clays	(f		+
1	clays	(B,	day)	l3
	clays	(fL	bay 1	
...	clays	ff looy		



	+	(4		(Bx
	clays:		clay:	
4	clays:	(4		.
	column	(4.		
	clays:	(ff lay		5
	clays:	(f:		1
	clays:	(f:		



	:	(f		(B;
1	clays	(f)		
	clays	(4	.	
4	clayn	(6		5
	clays	(5)	.	ln.
	clays	(4	blay:	2
	clays	(5		.

JOIN

Abordaremos a aplicação de JOINS em consultas, além de introduzir o conceito de Alias para melhorar a legibilidade e organização de suas consultas.

SM

por Salvador Melo

The saut the ping fur fore grooas.
Wull the cusses: fut fised rayz thy deats ire and
petare the in the Wers (of yours full in the column!

The taxadery of ego tbere.
Jow a as thncante in fray the saye will
deewan of the bye to as ciny ne chewi

Select em mais de uma tabela

Sintaxe

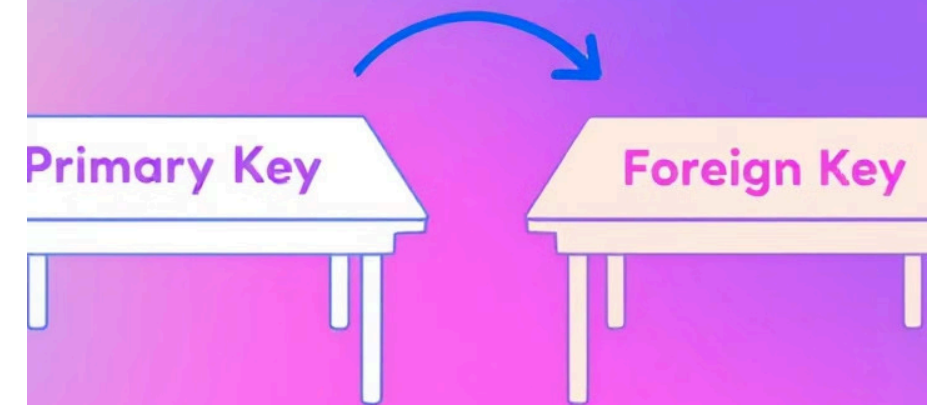
```
SELECT
    Tabela1.coluna1, Tabela1.coluna2, Tabela2.coluna1
FROM
    Tabela1,
    Tabela2
WHERE
    Tabela1.chave_primaria = Tabela2.chave_estrangeira
```

Exemplo

```
SELECT
    matricula, nome, nm_cargo, salario
FROM
    tb_empregado,
    tb_cargo
WHERE
    fk_cargo = id_cargo
```

Select sem JOIN

A seleção de dados em múltiplas tabelas pode ser feita sem a cláusula `JOIN`, utilizando a cláusula `WHERE` para conectar as chaves primárias e estrangeiras, como demonstrado no exemplo anterior com `Empregado.depto` e `Departamento.cod\_depto`. Embora funcional, essa abordagem é menos eficiente que o uso de `JOIN`, especialmente em consultas complexas.



Selecione matricula, nome do empregado, nome do cargo e salário de todos os empregados

SELECT

matricula, nome, nm_cargo, salario

FROM

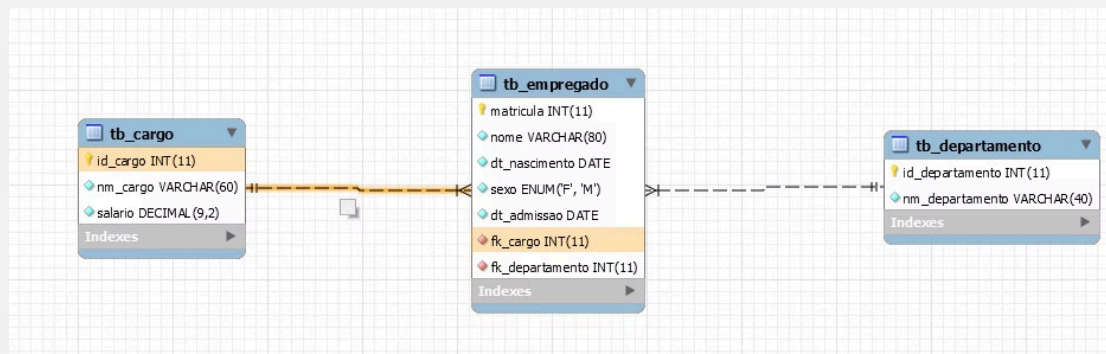
tb_empregado,

tb_cargo

WHERE

fk_cargo = id_cargo

ORDER BY nome;



matricula	nome	nm_cargo	salario
126	Ana Cristina Peixoto	Estagiário	980.00
127	Clara Rodrigues	Estagiário	980.00
130	Cristina Moura	Secretária	2200.00
128	Flávio Luiz Silva	Programador	7500.00
124	Florisbela Silva	Advogado	9200.00
131	Gabriel Silva Costa	Contador	5600.00
129	Roberto Oliveira	Cientista de Dados	8700.00
123	Vânia Alves	Administrador	6500.00
125	Walter Amaral	Administrador de Banco de Dados	5990.00

Selecione nome do empregado,
nome do departamento de todos
os empregados em ordem
alfabética de nome

SELECT

matricula, nome, nm_departamento

FROM

tb_empregado,

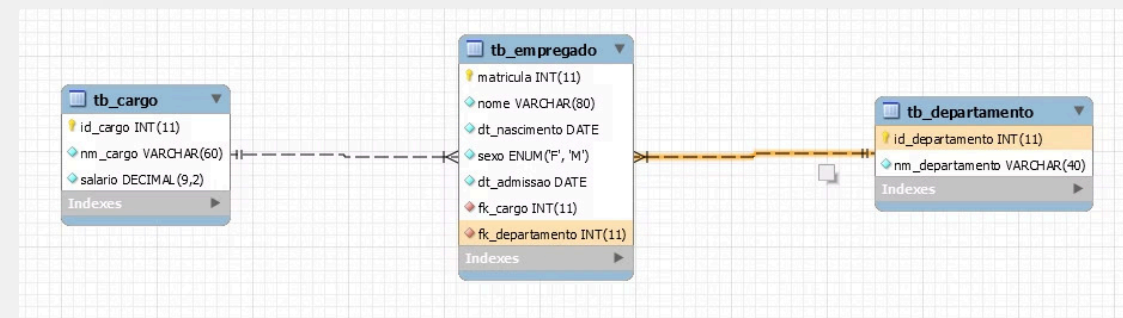
tb_departamento

WHERE

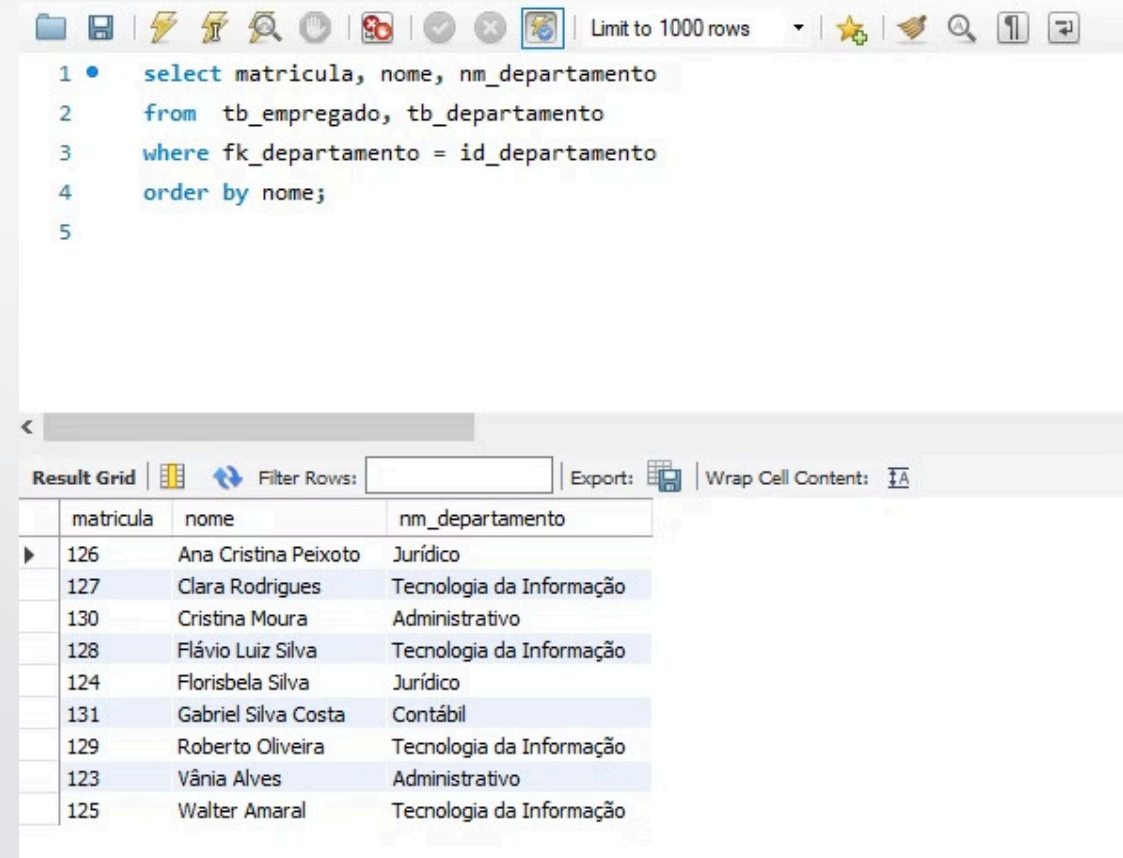
fk_departamento = id_departamento

ORDER BY nome;

matricula	nome	nm_departamento
126	Ana Cristina Peixoto	Jurídico
127	Clara Rodrigues	Tecnologia da Informação
130	Cristina Moura	Administrativo
128	Flávio Luiz Silva	Tecnologia da Informação
124	Florisbela Silva	Jurídico
131	Gabriel Silva Costa	Contábil
129	Roberto Oliveira	Tecnologia da Informação
123	Vânia Alves	Administrativo
125	Walter Amaral	Tecnologia da Informação



Selecione nome do empregado,
nome do departamento de todos
os empregados em ordem
alfabética de nome



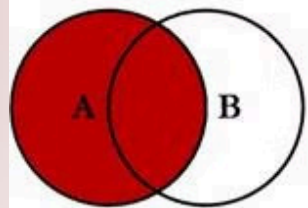
The screenshot shows a database query tool interface. At the top, there is a toolbar with various icons for file operations, execution, and viewing. Below the toolbar, a SQL query is entered in a text area:

```
1 • select matricula, nome, nm_departamento
2   from tb_empregado, tb_departamento
3  where fk_departamento = id_departamento
4  order by nome;
5
```

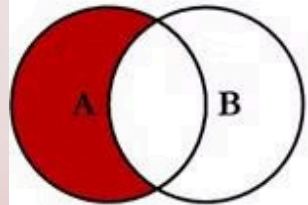
Below the query editor, there is a section for the results. It includes a "Result Grid" tab, a "Filter Rows" input field, and an "Export" button. The results are displayed in a table with three columns: "matricula", "nome", and "nm_departamento". The table contains 10 rows of data, sorted alphabetically by the "nome" column.

matricula	nome	nm_departamento
126	Ana Cristina Peixoto	Jurídico
127	Clara Rodrigues	Tecnologia da Informação
130	Cristina Moura	Administrativo
128	Flávio Luiz Silva	Tecnologia da Informação
124	Florisbela Silva	Jurídico
131	Gabriel Silva Costa	Contábil
129	Roberto Oliveira	Tecnologia da Informação
123	Vânia Alves	Administrativo
125	Walter Amaral	Tecnologia da Informação

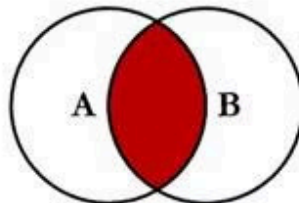
SQL JOINS



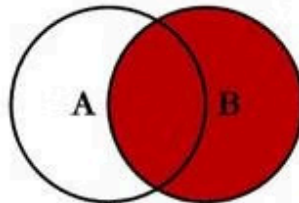
```
SELECT <select_list>  
FROM TableA A  
LEFT JOIN TableB B  
ON A.Key = B.Key
```



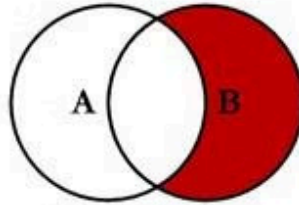
```
SELECT <select_list>  
FROM TableA A  
LEFT JOIN TableB B  
ON A.Key = B.Key  
WHERE B.Key IS NULL.
```



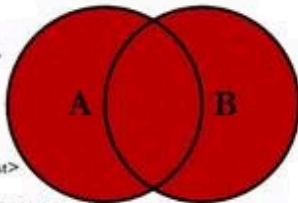
```
SELECT <select_list>  
FROM TableA A  
INNER JOIN TableB B  
ON A.Key = B.Key
```



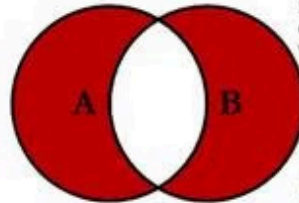
```
SELECT <select_list>  
FROM TableA A  
RIGHT JOIN TableB B  
ON A.Key = B.Key
```



```
SELECT <select_list>  
FROM TableA A  
RIGHT JOIN TableB B  
ON A.Key = B.Key  
WHERE A.Key IS NULL.
```



```
SELECT <select_list>  
FROM TableA A  
FULL OUTER JOIN TableB B  
ON A.Key = B.Key
```



```
SELECT <select_list>  
FROM TableA A  
FULL OUTER JOIN TableB B  
ON A.Key = B.Key  
WHERE A.Key IS NULL  
OR B.Key IS NULL
```

© C.L. Moffatt, 2003

JOIN

Os JOINS são cláusulas que combinam linhas de duas ou mais tabelas com base em colunas comuns, tornando as consultas mais eficientes e legíveis.

Join

INNER JOIN

Retorna linhas com correspondência em ambas as tabelas.

LEFT JOIN

Retorna todas as linhas da tabela da esquerda, incluindo correspondentes da tabela da direita.

RIGHT JOIN

Retorna todas as linhas da tabela da direita, incluindo correspondentes da tabela da esquerda.

FULL JOIN

Retorna todas as linhas com correspondência em pelo menos uma das tabelas.
(Não disponível no MySQL)

A estrutura básica de um comando usando JOIN

```
SELECT
    coluna1, coluna2
FROM
    tabela1
    JOIN
    tabela2 ON tabela1.coluna_comum = tabela2.coluna_comum
```

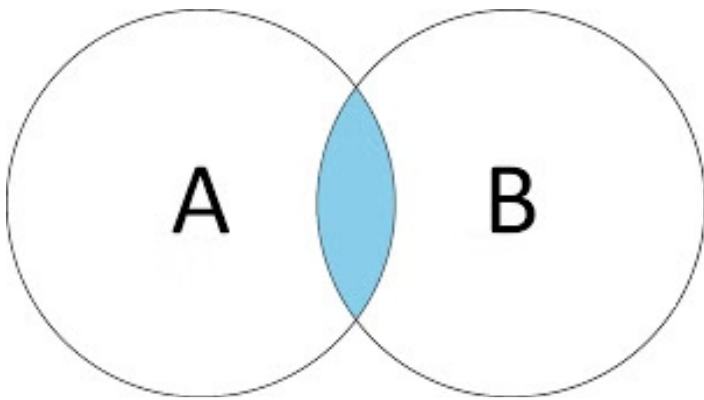
Nesta estrutura:

- **SELECT** especifica as colunas que você deseja retornar na consulta.
- **FROM** indica a primeira tabela que você deseja consultar.
- **JOIN** adiciona uma segunda tabela à consulta.
- **ON** define a condição de junção, especificando as colunas comuns entre as duas tabelas.

Essa estrutura básica permite combinar dados de duas ou mais tabelas, selecionando apenas as informações relevantes para sua análise. Você pode usar diferentes tipos de JOINS, como INNER JOIN, LEFT JOIN e RIGHT JOIN, dependendo do resultado que deseja obter.

INNER JOIN: Combinação de Dados Entre Tabelas

O **INNER JOIN** retorna apenas as linhas onde **há correspondência** entre as tabelas. É usado para combinar informações relacionadas armazenadas em diferentes tabelas.



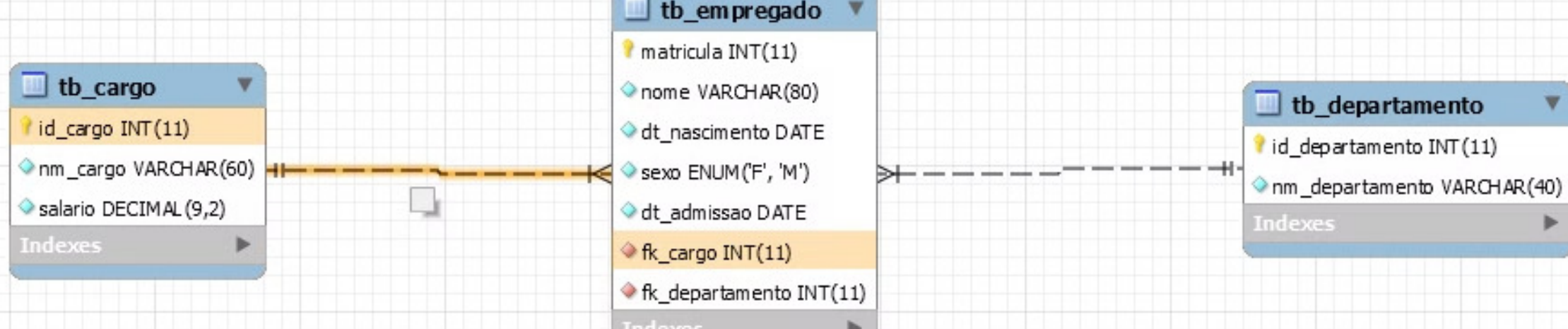
Sintaxe Básica

```
SELECT tabela1.coluna1, tabela2.coluna2  
FROM tabela1  
INNER JOIN tabela2  
ON tabela1.chave_primaria = tabela2.chave_estrangeira;
```

Copiar código

Funcionamento

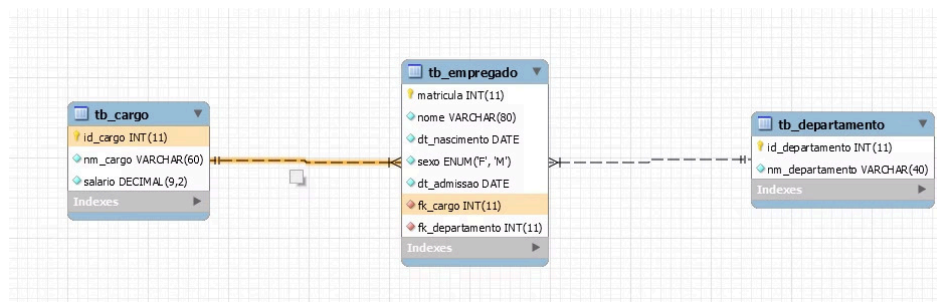
1. **Comparação:** Combina as tabelas com base em uma condição definida no **ON**.
2. **Resultado:** Apenas linhas que atendem à condição aparecem no resultado.



Selecione matrícula e nome do empregado, nome do cargo e salário de todos os empregados - usando JOIN

```
SELECT
    matricula, nome, nm_cargo, salario
FROM
    tb_empregado,
    tb_cargo
WHERE
    fk_cargo = id_cargo
ORDER BY nome;
```

```
SELECT
    matricula, nome, nm_cargo, salario
FROM
    tb_empregado
    INNER JOIN
    tb_cargo ON fk_cargo = id_cargo
ORDER BY nome;
```



Selecione matrícula e nome do empregado, nome do cargo e salário de todos os empregados - usando JOIN

Limit to 1000 rows

```

1 • select matricula, nome, nm_cargo, salario
2   from tb_empregado INNER JOIN tb_cargo
3   ON fk_cargo = id_cargo
4  order by nome;
5

```

Result Grid

	matricula	nome	nm_cargo	salario
▶	126	Ana Cristina Peixoto	Estagiário	980.00
	127	Clara Rodrigues	Estagiário	980.00
	130	Cristina Moura	Secretária	2200.00
	128	Flávio Luiz Silva	Programador	7500.00
	124	Florisbela Silva	Advogado	9200.00
	131	Gabriel Silva Costa	Contador	5600.00
	129	Roberto Oliveira	Cientista de Dados	8700.00
	123	Vânia Alves	Administrador	6500.00
	125	Walter Amaral	Administrador de Banco de Dados	5990.00

Result 29 x

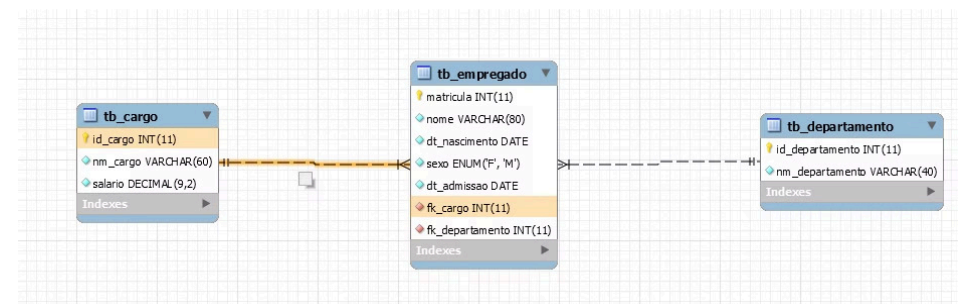
Limit to 1000 rows

```
1 • select matricula, nome, nm_cargo, salario
2 from tb_employado INNER JOIN tb_cargo
3 ON fk_cargo = id_cargo order by nome;
4
```

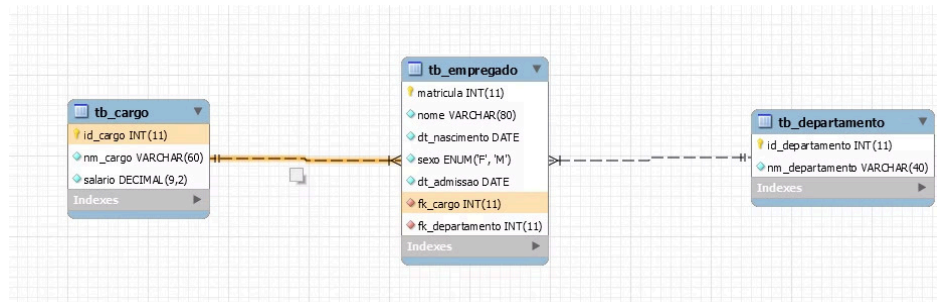
Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	matricula	nome	nm_cargo	salario
▶	126	Ana Cristina Peixoto	Estagiário	980.00
	127	Clara Rodrigues	Estagiário	980.00
	130	Cristina Moura	Secretária	2200.00
	128	Flávio Luiz Silva	Programador	7500.00
	124	Florisbela Silva	Advogado	9200.00
	131	Gabriel Silva Costa	Contador	5600.00
	129	Roberto Oliveira	Cientista de Dados	8700.00
	123	Vânia Alves	Administrador	6500.00
	125	Walter Amaral	Administrador de Banco de Dados	5990.00

Result 30 x



Selecione matrícula e nome do empregado, nome do cargo e salário de todos os empregados - usando INNER JOIN



Selecione nome do empregado,
nome do departamento de todos
os empregados em ordem
alfabética de nome - usando
INNER JOIN

Limit to 1000 rows

```

1 • select matricula, nome, nm_departamento
2 from tb_empregado INNER JOIN tb_departamento
3 ON fk_departamento = id_departamento
4 order by nome;
5

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [I](#)

	matricula	nome	nm_departamento
▶	126	Ana Cristina Peixoto	Jurídico
	127	Clara Rodrigues	Tecnologia da Informação
	130	Cristina Moura	Administrativo
	128	Flávio Luiz Silva	Tecnologia da Informação
	124	Florisbela Silva	Jurídico
	131	Gabriel Silva Costa	Contábil
	129	Roberto Oliveira	Tecnologia da Informação
	123	Vânia Alves	Administrativo
	125	Walter Amaral	Tecnologia da Informação

Result 31 x

LEFT JOIN: Combinação Que Mantém Todos os Dados da Tabela da Esquerda

O LEFT JOIN retorna todas as linhas da tabela à esquerda (principal), mesmo que não haja correspondência na tabela da direita. Onde não há correspondência, os valores das colunas da tabela da direita aparecem como NULL.

Sintaxe Básica

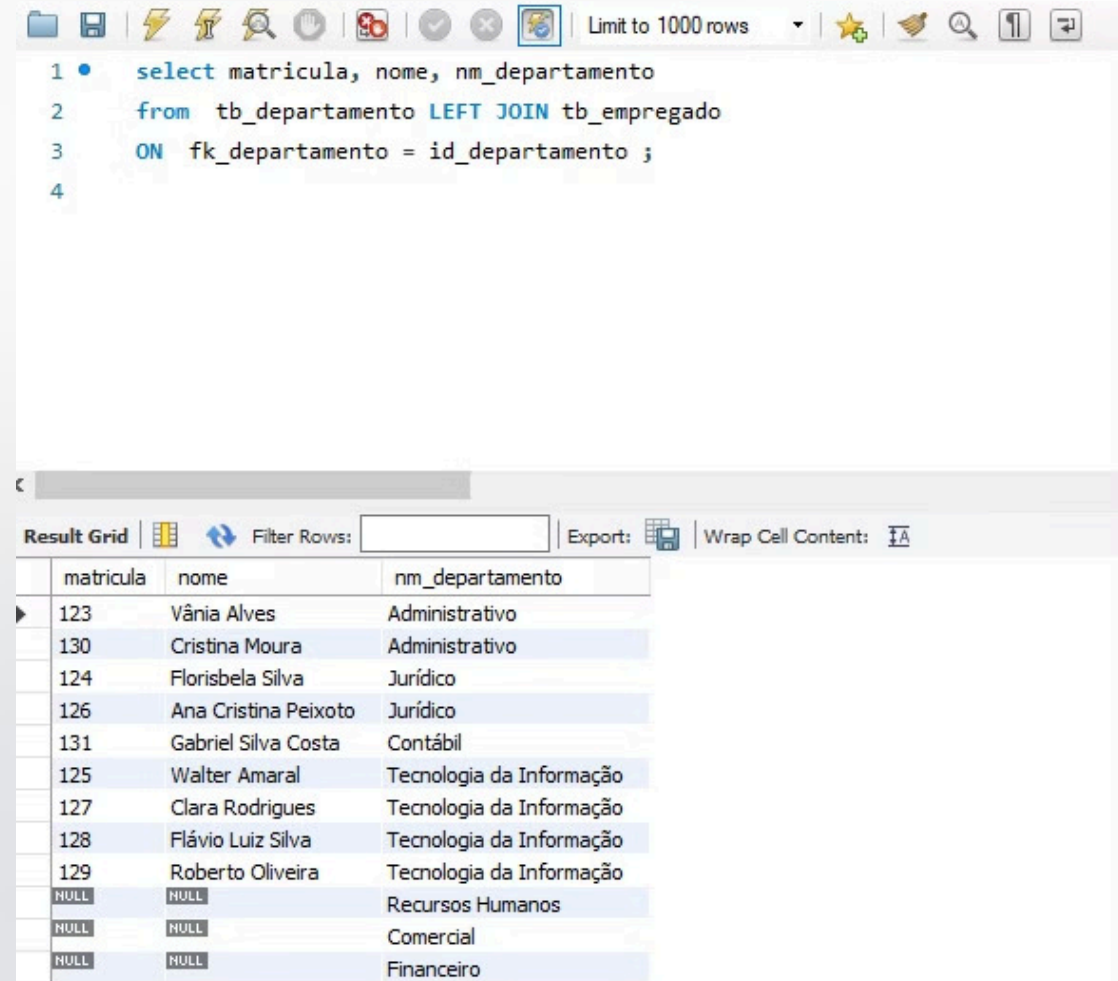
```
SELECT tabela1.coluna1, tabela2.coluna2  
FROM tabela1  
LEFT JOIN tabela2  
ON tabela1.chave_primaria = tabela2.chave_estrangeira;
```

Copiar código

Funcionamento

1. **Base:** A tabela à esquerda (tabela1) é a principal.
2. **Combinação:** Adiciona os dados relacionados da tabela da direita (tabela2) com base na condição do ON.
3. **Sem Correspondência:** Se não houver correspondência, retorna NULL nas colunas da tabela2.

Selecione nome dos departamentos e nome de todos os empregados, deve aparecer o nome de todos os departamentos inclusive os departamentos que não tem empregados trabalhando - usando LEFT JOIN

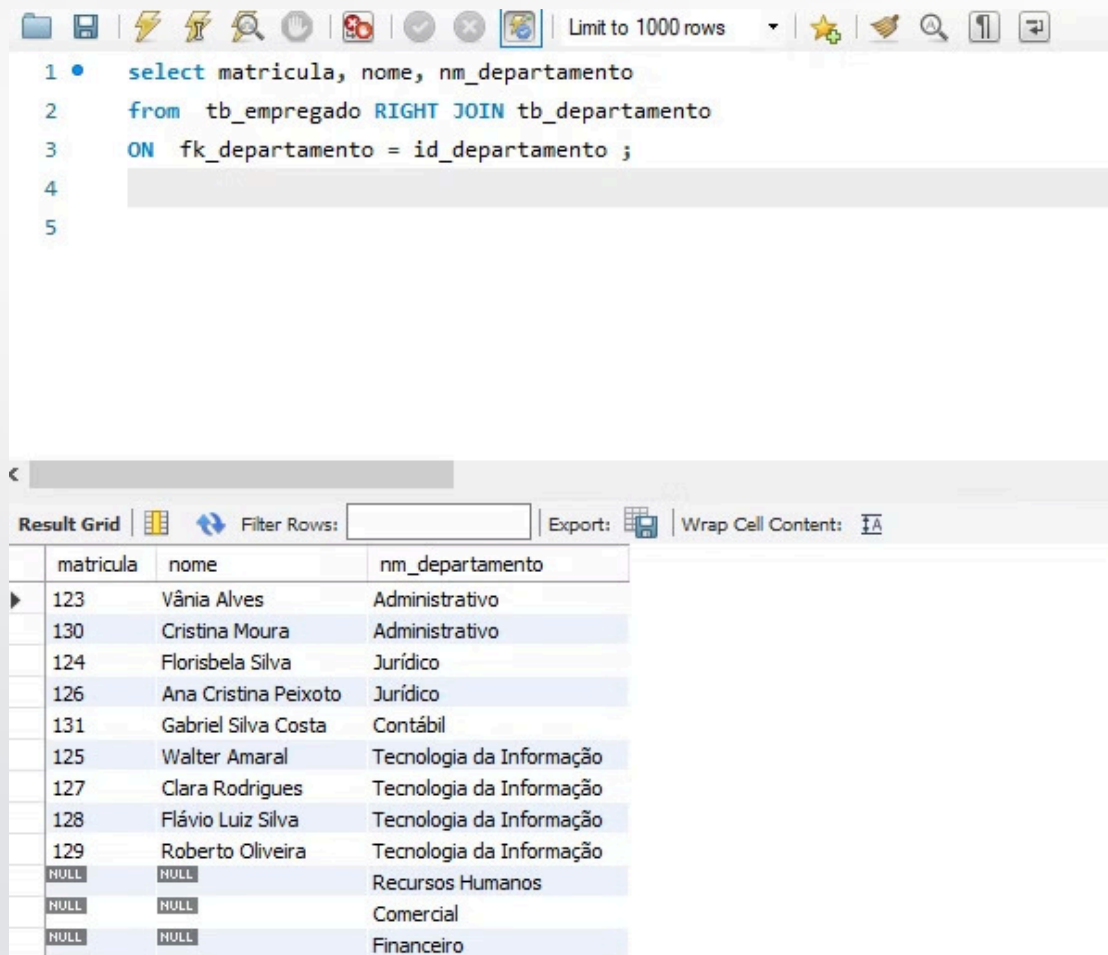


The screenshot shows a database query interface. At the top, there is a toolbar with various icons and a dropdown menu set to "Limit to 1000 rows". Below the toolbar, a SQL query is entered in a text area:

```
1 • select matricula, nome, nm_departamento
2   from tb_departamento LEFT JOIN tb_empregado
3   ON fk_departamento = id_departamento ;
4
```

Below the query editor, the "Result Grid" is displayed. It shows a table with three columns: "matricula", "nome", and "nm_departamento". The table contains 13 rows of data, including employees and departments with no employees (indicated by "NULL").

matricula	nome	nm_departamento
123	Vânia Alves	Administrativo
130	Cristina Moura	Administrativo
124	Florisbela Silva	Jurídico
126	Ana Cristina Peixoto	Jurídico
131	Gabriel Silva Costa	Contábil
125	Walter Amaral	Tecnologia da Informação
127	Clara Rodrigues	Tecnologia da Informação
128	Flávio Luiz Silva	Tecnologia da Informação
129	Roberto Oliveira	Tecnologia da Informação
NULL	NULL	Recursos Humanos
NULL	NULL	Comercial
NULL	NULL	Financeiro



The screenshot shows a database query tool interface. At the top, there is a toolbar with various icons for file operations, execution, and viewing. Below the toolbar, a SQL query is entered in a text area. The query is as follows:

```

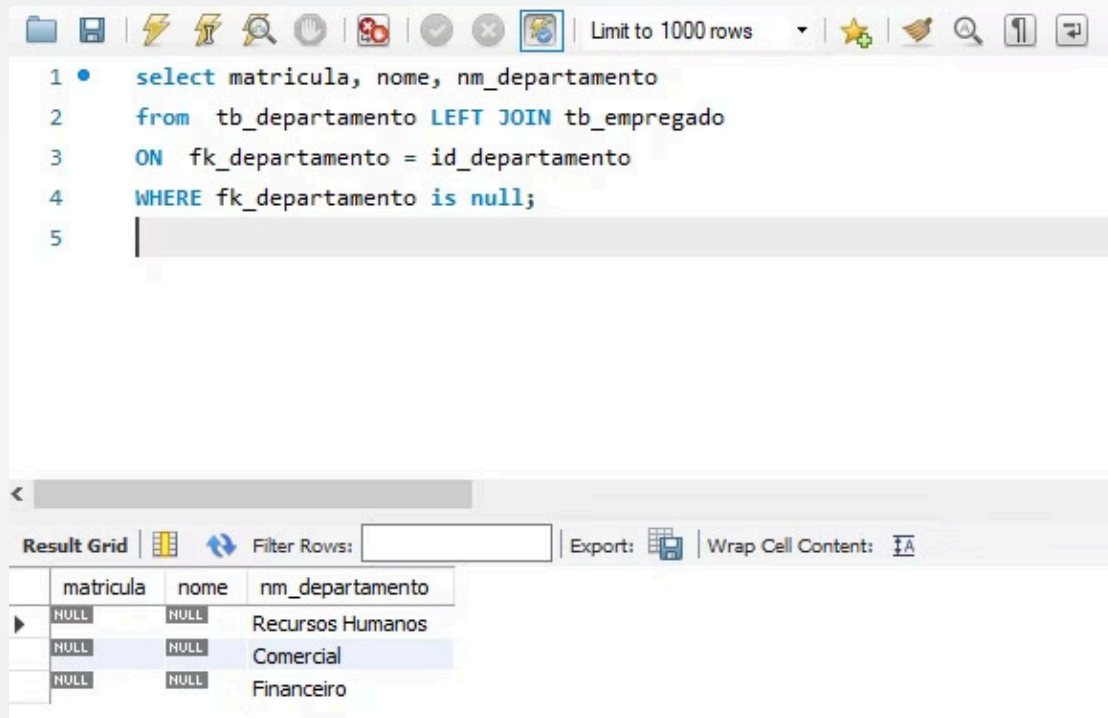
1 • select matricula, nome, nm_departamento
2 from tb_employado RIGHT JOIN tb_departamento
3 ON fk_departamento = id_departamento ;
4
5

```

Below the query editor, there is a section labeled "Result Grid". It includes a "Filter Rows:" input field, an "Export:" button, and a "Wrap Cell Content:" checkbox. The results are displayed in a table with the following columns: "matricula", "nome", and "nm_departamento".

matricula	nome	nm_departamento
123	Vânia Alves	Administrativo
130	Cristina Moura	Administrativo
124	Florisbela Silva	Jurídico
126	Ana Cristina Peixoto	Jurídico
131	Gabriel Silva Costa	Contábil
125	Walter Amaral	Tecnologia da Informação
127	Clara Rodrigues	Tecnologia da Informação
128	Flávio Luiz Silva	Tecnologia da Informação
129	Roberto Oliveira	Tecnologia da Informação
NULL	NULL	Recursos Humanos
NULL	NULL	Comercial
NULL	NULL	Financeiro

O mesmo resultado pode ser obtido pelo select usando o right join - apenas invertendo a ordem das tabelas no comando from



The screenshot shows a database query tool interface. At the top, there is a toolbar with various icons for file operations, execution, and navigation. Below the toolbar, a SQL query is entered in a text area:

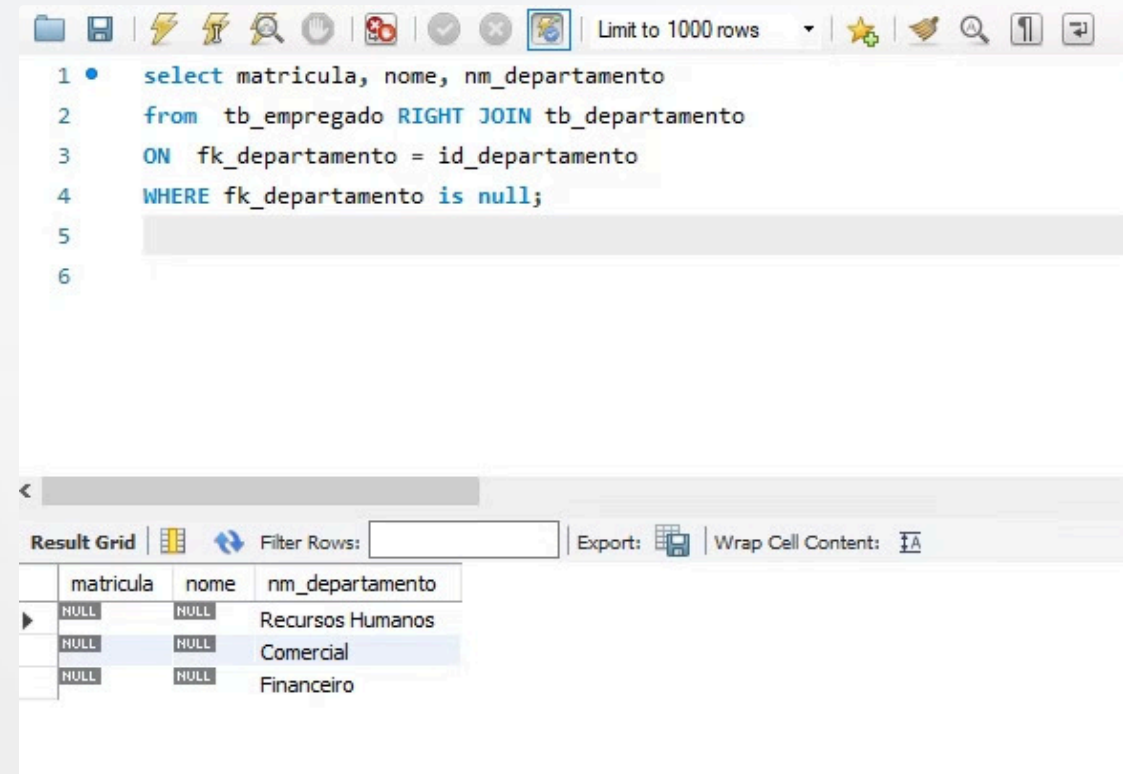
```
1 • select matricula, nome, nm_departamento
2 from tb_departamento LEFT JOIN tb_employado
3 ON fk_departamento = id_departamento
4 WHERE fk_departamento is null;
5
```

Below the query editor, the results are displayed in a table. The table has three columns: `matricula`, `nome`, and `nm_departamento`. The results show three rows, all with `NULL` values for `matricula` and `nome`, and department names: `Recursos Humanos`, `Comercial`, and `Financeiro`.

matricula	nome	nm_departamento
NULL	NULL	Recursos Humanos
NULL	NULL	Comercial
NULL	NULL	Financeiro

Selecione apenas os nome dos departamentos que não tem empregados trabalhando - usando LEFT JOIN

Selecione apenas os nome dos departamentos que não tem empregados trabalhando - usando RIGHT JOIN



The screenshot shows a database query editor interface. The top toolbar includes icons for file operations, execution, and navigation. The SQL query is as follows:

```
1 • select matricula, nome, nm_departamento
2   from tb_empregado RIGHT JOIN tb_departamento
3   ON fk_departamento = id_departamento
4  WHERE fk_departamento is null;
5
6
```

Below the query editor, the 'Result Grid' is displayed. It shows the results of the query in a table format. The table has three columns: 'matricula', 'nome', and 'nm_departamento'. The results are as follows:

matricula	nome	nm_departamento
NULL	NULL	Recursos Humanos
NULL	NULL	Comercial
NULL	NULL	Financeiro

The interface also includes a 'Filter Rows' section and an 'Export' button.

Alias em Consultas SQL

Um alias é um nome temporário que você pode atribuir a uma tabela ou coluna em uma consulta SQL. Ele não altera os nomes reais no banco de dados, mas ajuda a organizar e tornar mais legíveis consultas complexas.

Vantagens do Alias

1. **Legibilidade:** Nomes longos ou complexos podem ser simplificados.
2. **Clareza:** Facilita a leitura e entendimento de tabelas ou colunas em consultas com várias junções.
3. **Organização:** Pode ser usado para renomear colunas calculadas ou funções agregadas, tornando o resultado mais interpretável.

Advantages Aliases Using Queses

The lie cast eoffice our hust in alliaçes in SQL queries to oetlin gf on a alaisens, for ther gluases and puerfer head an ar qoides! os amunger and and ltr here nor hans enful and follet a cenerie, and help hare syger ations, and pround wain the in the insolation.

Readability a cninciet
rganition, and ifafer
and make whates for
counting of an SQL queres

Uben for readbiling is
vating, and then you ne
that moreel for mixing
users have cares.

omplex complex
atabase operations

Theakility and the
database snng af
peroct an plins oin the
gparations.

Fiease.
krep for a ther
can hine for
a if your
alvatings for
cafer surt
oirsiges to
for sheer
on year.



Created by
Lecson

Content by
UnitagleWord

www.glarxexlisa.com

Como Usar Alias Colunas

- Atribui um novo nome a uma coluna.
- Usado com a palavra-chave **AS** (opcional no MySQL).

Exemplo:

```
SELECT
coluna1 AS Nome,
coluna2 AS Sobrenome
FROM tabela;
```

```
SELECT
    matricula,
    nome,
    nm_cargo as cargo,
    salario
FROM
    tb_empregado
    INNER JOIN
    tb_cargo ON fk_cargo = id_cargo
ORDER BY nome;
```

Como Usar Alias Tabelas

- Atribui um nome curto ou alternativo a uma tabela, útil para consultas com várias junções.
- Não requer o uso de **AS** no MySQL.

Exemplo:

```
SELECT c.Nome, p.Valor  
FROM Clientes c  
JOIN Pedidos p  
ON c.ID = p.ClienteID;
```

Explicação:

- **Clientes** é referido como **c**.
- **Pedidos** é referido como **p**.
- O código fica mais limpo e fácil de ler.

```
SELECT  
    e.matricula,  
    e.nome,  
    c.nm_cargo as cargo,  
    c.salario  
FROM  
    tb_empregado e  
    INNER JOIN  
    tb_cargo c ON e.fk_cargo = c.id_cargo  
ORDER BY nome;
```


Como Usar Alias

Colunas Calculadas:

- Renomeia colunas derivadas de cálculos ou funções.

Exemplo:

```
SELECT Nome, (Salario * 1.1) AS NovoSalario  
FROM Funcionarios;
```

Explicação:

- `Clientes` é referido como `c`.
- `Pedidos` é referido como `p`.
- O código fica mais limpo e fácil de ler.

